

Algorithm Engineering Lab 0

Md. Atiqur Rahman

January 2024

Instructions

- Submit a report in latex, giving solutions to all the following tasks.
- For offline evaluation, you can show your lab instructor the solution in a notebook (not .ipynb, physical notebook).
- To show your implementation (only implement something when your are asked to) online, submit .ipynb file with your report and in offline evaluation show that .ipynb file to your lab instructor.

Task 1

Sort and draw the following functions with [desmos](#) in increasing order of asymptotic (big-O) complexity with justification:

- $f_1(n) = n^{0.999} * \log_2(n)$
- $f_2(n) = \binom{n}{2}$
- $f_3(n) = (1000001 * 10^{-6})^n$
- $f_4(n) = n!$
- $f_5(n) = 2^{(10*10^7)}$
- $f_6(n) = n * \sqrt{n}$
- $f_7(n) = n^{\sqrt{n}}$
- $f_8(n) = \sum_{i=1}^n (i + 1)$

Task 2

Find the asymptotic (big-O) complexity of the below codes with justification:

1. Find only the complexity for **find3RDelement** function.

```
def find3RDelement(a):  
  
    flag = 2  
  
    for i in range(len(a)):  
        if i == flag : return a[i]  
  
    print (find3RDelement ([1,2,7,4,5]))
```

2. Find only the complexity for **doingSomethingImportant**.

```
import numpy as np

def doingSomethingImportant(p2, sr, sc, prev, new):

    row = len(p2)

    col = len(p2[0]) if len(p2) > 0 else 0

    if sr < 0 or sr >= row or sc < 0 or sc >= col:
        return

    if p2[sr][sc] != prev:
        return

    p2[sr][sc] = new

    doingSomethingImportant(p2, sr - 1, sc, prev, new)

    doingSomethingImportant(p2, sr + 1, sc, prev, new)

    doingSomethingImportant(p2, sr, sc + 1, prev, new)

    doingSomethingImportant(p2, sr, sc - 1, prev, new)


a2 = [
    [1, 1, 1, 1, 1, 1, 1, 1],
    [1, 1, 1, 1, 1, 1, 0, 0],
    [1, 0, 0, 1, 1, 0, 1, 1],
    [1, 2, 2, 2, 2, 0, 1, 0],
    [1, 1, 1, 2, 2, 0, 1, 0],
    [1, 1, 1, 2, 2, 2, 2, 0],
    [1, 1, 1, 1, 1, 2, 1, 1],
    [1, 1, 1, 1, 1, 2, 2, 1]
]

x,y = 4,4

prev = a2[x][y]

new = 3

doingSomethingImportant(a2, x, y, prev, new)

a2 = np.array(a2)

print(a2)
```

3. Find only the complexity for **doingSomethingImportant** function.

```
import heapq

def addEdge(adj, u, v, w):

    adj[u].append((v, w))

    adj[v].append((u, w))

    return adj


def doingSomethingImportant(adj, V, sc):

    pq = []
    heapq.heappush(pq, (0, sc))

    cost = [float('inf')] * V
    cost[sc] = 0

    while pq:

        d, u = heapq.heappop(pq)

        for v, weight in adj[u]:

            if cost[v] > cost[u] + weight:

                cost[v] = cost[u] + weight

                heapq.heappush(pq, (cost[v], v))

    return cost


g = [[] for _ in range(V)]
V = 3

g = addEdge(g, 0, 1, 4)
g = addEdge(g, 0, 2, 5)
g = addEdge(g, 1, 2, 4)

print(
    [
        (node, cost)
        for node, cost in enumerate(doingSomethingImportant(g, V, 0))
    ]
)
```

Task 3

Propose an algorithm which will print co-ordinate of a peak in a 2D matrix. A peak in a matrix is a location with the property that its four neighbors (north, south, east, and west) have value less than or equal to the value of the peak. If the following matrix is given as input you need to output (1,1).

```
0 1 0
1 2 1
0 1 0
```

You don't need to do anything if there is no element left to compare. For the following matrix, answer is (0,1) or (2,1) since there can be multiple correct answers.

```
0 1 0
0 0 0
0 0 0
```

Your algorithm should have lower or equal to $O(row * \log(column))$ time complexity. For better marks implement your algorithm in python and test with different test cases. Submit your implementation with your report.